



NDCAK DIGITAL TOURNAMENT MANAGEMENT SYSTEM

Product Requirements Document (PRD) + Technical Implementation Blueprint

Notre Dame College Association of KUET
Indoor Games Championship Platform

Purpose of this document

This is an implementation-ready specification for a coding agent or engineering team. It defines product behavior, user flows, permissions, data model, UI/UX rules, technology choices, backend architecture, edge cases, security, performance, testing, and acceptance criteria. It is intentionally more precise than the current conceptual Figma screens.

Version 1.0 | 18 September 2026

Status: Approved product direction; game-specific rules remain configurable

Document Control

Item	Value
Product	NDCAK Digital Tournament Management System
Document	Product Requirements Document + Technical Blueprint
Version	1.0
Primary event	NDCAK Indoor Games Championship
Primary users	Participants, Super Admin, Score Operators
Implementation stack	Next.js App Router + TypeScript + Drizzle + Supabase PostgreSQL/Auth/Realtime + Resend + Vercel
Product principle	Participant experience must remain one-time, simple and account-free; operational complexity stays with staff.

Configuration disclaimer

Dates, fees, slot counts, payment receiving numbers, schedules, scoring rules and example participant data visible in existing concept screenshots are NOT to be hardcoded. They are event configuration values unless the organizing committee explicitly finalizes them.

Priority Legend

Priority	Meaning
P0	Required for the first production event.
P1	Strongly recommended if time allows; architecture should support it.
P2	Future enhancement; do not block V1.

Contents

- 1. Executive Summary
- 2. Product Vision, Objectives and Non-Goals
- 3. User Roles and Experience Boundaries
- 4. UX and Visual Design System
- 5. Public Website Information Architecture
- 6. Participant Registration and Manual Payment Verification
- 7. Super Admin Registration Review
- 8. Tournament, Games and Participant Model
- 9. Score Operator Accounts and Flexible Assignments
- 10. Match Operations and Game-Specific Scoring
- 11. Tournament Progression and Results
- 12. Super Admin Operations, Analytics and Reporting
- 13. Notifications and Email Automation
- 14. Pagination, Search, Filtering and Data Management
- 15. Responsive, Accessibility and Performance Requirements
- 16. Technical Architecture and Stack
- 17. Database Design
- 18. Authorization, Security, Concurrency and Auditability
- 19. Application Routes and Screen Specifications
- 20. Codebase Architecture and Engineering Conventions
- 21. API / Server Action Contract Guidance
- 22. Testing and Quality Strategy
- 23. Deployment, Environments and Operations
- 24. Delivery Phases
- 25. Acceptance Criteria
- 26. Edge Cases and Failure Handling
- 27. Future Extensions and Explicit Non-Goals
- 28. Implementation Checklist and Definition of Done

1. Executive Summary

NDCAK Digital Tournament Management System is a centralized web platform for operating the full lifecycle of an NDCAK indoor games event. It combines a public event website, a one-time participant registration and manual payment verification flow, a Super Admin control center, and a restricted Score Operator workspace for game-wise / participant-wise result entry.

The product is not a generic esports portal and should not look or behave like one. The public side should feel like a modern university-association event website: clean, credible, energetic, responsive and easy to understand. Staff dashboards should be operational tools first: information-dense where necessary, but restrained, legible and task-focused.

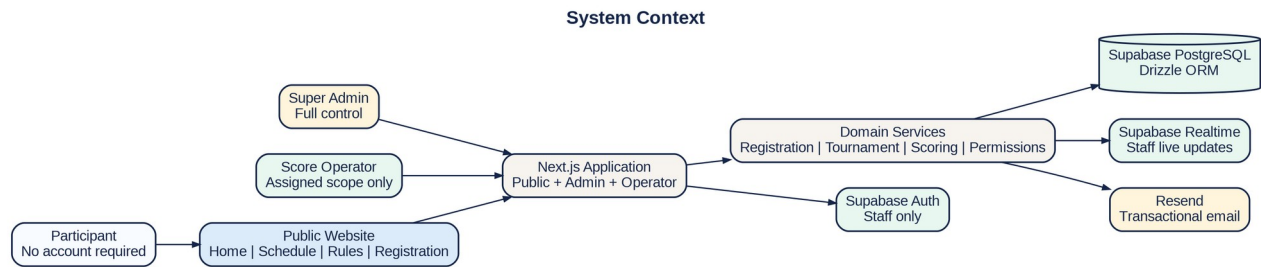


Figure 1. System context and major external services.

Core product rule

Keep participant interaction simple

A participant selects every desired game during the initial registration form, reviews details, submits payment information once, and waits for admin verification. There is no participant account, no later game-registration phase, and no tournament-management dashboard for participants.

Core operational rule

A game can contain many participants, many rounds and many matches, and multiple operators may work on the same game. One operator can also be assigned to multiple games. Admin assignment must be flexible enough to grant an operator access to the whole tournament, one or more games, selected rounds, selected matches, or selected participant entries.

2. Product Vision, Objectives and Non-Goals

2.1 Vision

Replace fragmented forms, spreadsheets, manual messaging and ad-hoc score tracking with a single trusted tournament operations platform that is simple for students and controlled for organizers.

2.2 Product objectives

- Provide a polished public event website with event information, schedule, rules, prizes and registration.
- Allow a student to register for multiple eligible game segments in one straightforward flow.
- Capture payment method and transaction ID, then keep the request pending until manual Super Admin verification.
- Give Super Admin full visibility into registrations, payment status, games, participants, operators, assignments, matches, score activity and analytics.
- Allow Super Admin to create operator accounts and assign flexible scopes without exposing unrelated participant/payment data.
- Provide game-specific score-entry experiences rather than forcing every game into one Player A / Player B numeric format.
- Support matches with two or more competitors; the data model must not hardcode exactly two players.
- Update Super Admin views near real time as operators submit results.
- Preserve an audit trail for sensitive actions and score/result changes.
- Remain responsive, fast and usable on mobile, tablet and desktop using free-tier infrastructure for the expected event scale.

2.3 Success indicators

Area	Success indicator
Registration	A student completes the full registration/payment submission without staff assistance.
Operations	Admin can verify, approve/reject and find registrations quickly with server-side filters and pagination.
Score entry	An operator can find an assigned match and submit a valid result in a few interactions.
Reliability	No duplicate confirmed slot, inconsistent bracket progression or unauthorized score edit under concurrent use.
Performance	Primary screens feel responsive on typical campus/mobile internet and do not load excessive data.
Auditability	Admins can identify who changed a match/registration, what changed and when.

2.4 Explicit non-goals for V1

- No participant login/account/password system.
- No automatic bKash/Nagad/Rocket API payment verification; payment verification is manual by Super Admin.
- No participant-side match management, team management or score submission.
- No native mobile application; the responsive web application is the product.
- No paid infrastructure is required initially unless actual usage demonstrates a need.
- No attempt to build a universal tournament SaaS in V1. Build for NDCAK first, while keeping the domain model configurable.
- No unnecessary social feed, chat, gamified profiles, animated esports clutter, or decorative widgets that do not support the event flow.

3. User Roles and Experience Boundaries

Role	Authentication	Primary responsibility	Data boundary
Participant / public visitor	None	View event information; submit one registration and payment reference.	Own submitted information only on immediate confirmation screen/email; no staff data.
Score Operator	Supabase Auth	Enter/update results for assigned scope; view assigned games/matches/participants.	No payment details. No unrelated games/participants. No operator administration.
Super Admin	Supabase Auth	Configure event, verify registrations, create operators, assign scopes, oversee matches/results, analytics and audit.	Full application access.

3.1 Participant experience boundary

- One registration journey only: form -> review -> payment information -> submitted/pending -> email after decision.
- Game selection happens inside the initial form. Selected game entries are created automatically in the backend.
- No additional action is required after submission unless an organizer contacts the participant following rejection/problem.
- The same generated registration code can remain the stable identifier from submission through approval. It is valid for check-in only when registration status is CONFIRMED.
- Participants should never see internal operator assignment, audit logs, payment verification controls, internal notes or unpublished results.

3.2 Operator experience boundary

- Operator accounts are created/deactivated by Super Admin; there is no public operator signup.
- Operator navigation is generated from effective permissions, not from a hardcoded single-game role.
- Operators see only minimum participant identity needed for match operations (for example name, registration code/student ID if required, assigned game and match context).
- Operators do not see payment method, transaction ID, personal phone/email unless a specific operational requirement is later approved.
- Completed/finalized result correction is controlled; normal operators cannot silently rewrite historical results.

3.3 Super Admin experience boundary

- All critical configuration and staff management lives under the Super Admin area.
- Admin can override tournament state when necessary, but every override must produce an audit event.
- Admin tables must use search/filter/pagination and must not depend on loading the entire dataset into the browser.
- Admin must be able to reassign work quickly on event day without editing database records manually.

4. UX and Visual Design System

4.1 Design direction

The visual direction is institutional sports-event design, not a neon gaming template. The association logo is the source of brand identity. Use dark navy as the main foundation, warm gold as the primary brand/action accent, association green for success/live states, and warm ivory/off-white for high-contrast text. Red is reserved for destructive/error/rejection states.

Token	Recommended value	Usage
Deep Navy	#16264A	Primary brand/background/navigation.
Warm Gold	#E4C27A	Primary CTA, active navigation, key highlights.
Association Green	#118A4A	Success, confirmed, live/valid states.
Warm Ivory	#F6F1E6	Primary text on dark backgrounds / warm light surfaces.
Surface Navy	#1E2D52	Cards, panels, inputs on dark layouts.
Muted Gold	#C9A861	Secondary emphasis and pending/review.
Slate Text	#A8B0C5	Secondary text and labels on dark UI.
Alert Red	#C94C5B	Reject, error, delete/cancel only.

Brand accuracy note

These values are visually derived from the supplied association logo for the web system. If NDCAK has an official brand

guideline with exact colors, that guideline takes precedence.

4.2 Typography

Role	Font guidance	Rules
UI / body	Inter	Default for forms, tables, dashboards, buttons and long text. 14-16px public body; 13-15px dense staff UI.
Event display	Barlow Condensed (or similarly clean condensed sans)	Use sparingly for hero/event titles and section moments. Never use for long paragraphs or dense tables.
Association identity	Use supplied logo artwork	Do not recreate the wordmark with a substitute typeface.

4.3 UI principles

- Clarity over decoration: every visible element should help orientation, decision-making or action.
- Consistent spacing: use an 8px base spacing rhythm and a constrained container width on public pages.
- Primary action should be visually obvious; do not place multiple equally loud CTAs together.
- Use icons as support, not as replacements for clear labels.
- Do not use emojis as production UI icons. Use Lucide or purpose-built SVG icons.
- Avoid giant all-caps blocks in forms/admin pages. Reserve condensed uppercase typography for event display moments.
- No random gradients, glows, neon lime, oversized statistic walls, or decorative cards just because space exists.
- Use real event/game imagery only where it improves the public story. Use licensed/free-to-use images, optimize them, and avoid imagery in operational dashboards unless necessary.

4.4 Responsive behavior

- Public pages are mobile-first. Registration must be effortless on common phones.
- Operator dashboard is mobile/tablet-first because staff may move around the venue.
- Admin dashboard is desktop-optimized but must remain usable on tablet/mobile. Dense tables may become stacked cards or horizontally scrollable views with key columns preserved.
- No layout should depend on hover-only interactions.
- Touch targets should be at least approximately 44x44 CSS pixels for important controls.

5. Public Website Information Architecture

The public website should look like a real event microsite with a small, focused navigation and strong content hierarchy. Staff operations should not dominate the public navigation.

Route / Page	Purpose	P0 content
/	Event landing	Event title, concise value proposition, date/venue, registration CTA, game summary, important deadlines, prize/highlight summary, organizer identity.
/register	One-time registration	Student information + game selection with capacity/fee visibility.
/register/review	Review	Read-only summary of entered info, selected games and fee; edit/back action.
/register/payment	Payment submission	Payment method selection, receiving account/instructions, expected amount, transaction ID.
/register/submitted	Pending result	Registration code, pending verification status, submitted details, receipt email confirmation. No "You are confirmed" language.
/schedule	Schedule	Day/game/round schedule with responsive filters or tabs.
/rulebook	Rules	General rules plus game-specific sections.
/results (optional P1)	Public results	Only organizer-approved live/completed results, brackets or standings.

5.1 Public navigation

Recommended primary navigation: Home, Register, Schedule, Rulebook, and (if enabled) Results. Staff Login should be a subtle footer link or dedicated /staff/login route rather than a prominent public "Admin" item.

5.2 Content configuration

- Event name, description, dates, venue, registration open/close timestamps and reporting instructions are data/configuration, not hardcoded JSX strings.
- Each tournament game exposes name, icon/image, fee, capacity, short description, registration availability and rule content.
- Public slot counts may show confirmed + pending reservations subtracted from capacity to prevent misleading availability.
- Public pages should not expose internal participant lists unless explicitly enabled by Admin.

6. Participant Registration and Manual Payment Verification

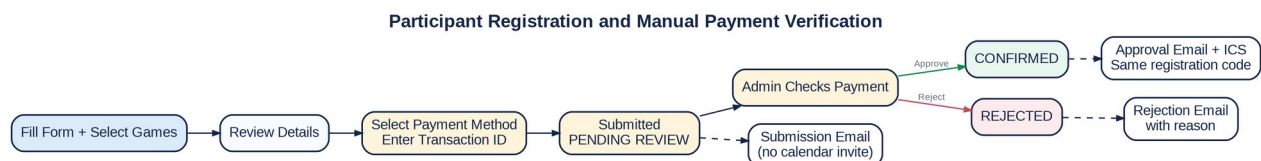


Figure 2. Final participant registration flow.

6.1 Registration form (P0)

Field	Validation / behavior
Full name	Required; trimmed; reasonable length limit.
Student ID	Required; normalized; active duplicate submission rules enforced.
Email	Required; valid email; used for submission/approval/rejection communication.
Phone	Required; Bangladesh-friendly validation but avoid overly rigid formatting.
Department	Required selection from configured list.
Academic year	Required selection from configured list.
Selected games	At least 1; no more than configured maximum; only open games with available capacity.

The game-selection section must clearly show each game's fee and remaining capacity. The total fee updates immediately as selections change. All calculations are repeated server-side; client totals are never trusted.

6.2 Review step (P0)

- Display all participant fields, selected games, per-game fee where useful, and total expected payment.
- Provide a single Back/Edit action and a single Proceed to Payment primary action.
- No database confirmation occurs merely because the user reached the review page.

6.3 Payment submission (P0)

- Admin configures enabled payment methods (for example bKash, Nagad, Rocket), display name, receiving account/number and instructions.
- Participant selects one method, sends payment outside the website, then enters the transaction ID/reference.
- System records expected amount, selected payment method, a snapshot of the receiving number/account shown at submission, and normalized transaction ID.
- Transaction ID should be normalized (trim/case normalization) and checked for duplicate use within the same provider. Duplicate detection should block or flag submission rather than silently accepting it.
- V1 does not request a payment screenshot and does not integrate a payment gateway/API unless the committee later changes scope.

6.4 Atomic submission and slot reservation

Final submission must be transactional. The server rechecks registration window, per-person selection limit, game status and capacity. It then creates the participant/registration records, pending game entries, payment submission and pending slot reservations in one database transaction. If any required game no longer has capacity, nothing is partially created and the user is asked to revisit selection.

```
BEGIN
  validate registration window and input
  upsert/create participant identity
  create registration(status=PENDING_REVIEW)
  create registration_game_entries(status=PENDING)
  reserve capacity for each selected game
  create payment(status=SUBMITTED)
  write audit/system event
COMMIT
```

6.5 Submission response

- Generate a human-readable registration code such as REG-2026-000147. Use the same code through pending and confirmed states to avoid confusing participants with multiple identifiers.
- Show "Registration Submitted" / "Pending Verification", never "Registration Confirmed" before admin approval.
- Send a submission acknowledgment email with registration code, selected games, expected amount/payment reference summary, and a clear statement that approval is pending.
- Do NOT attach the final calendar invite at this stage; send it after approval.

6.6 Registration state model

State	Meaning	Participant-facing wording
PENDING_REVIEW	Payment information submitted; waiting for admin.	Submitted - payment verification pending.
CONFIRMED	Admin verified and approved.	Registration confirmed.
REJECTED	Admin rejected for a recorded reason.	Registration could not be confirmed; see reason/contact organizer.
CANCELLED (optional)	Organizer/admin cancellation after confirmation.	Cancelled by organizer.

7. Super Admin Registration Review

7.1 Pending approval queue

Pending registrations are a dedicated operational queue, not merely a status column in a giant table. Default view should show oldest pending submissions first or allow sorting.

Capability	Requirement
Search	Name, student ID, registration code, email or transaction ID.
Filters	Status, game, department, payment method, submission date range.
Sorting	Submitted time, name, amount; default oldest pending first.
Pagination	Server-side, default 25 rows/page; preserve query filters in URL.
Quick status tabs	Pending, Confirmed, Rejected, All.
Review action	Open focused detail page/drawer without losing list filters.

7.2 Registration review detail

- Participant identity: name, student ID, department/year, email, phone.
- Selected games and expected fee calculation.
- Payment method, receiving account/number snapshot, expected amount, transaction ID and submission timestamp.
- Registration status and audit timeline.
- Approve and Reject actions. Reject requires a reason; optional internal note is allowed.
- Admin manually checks the transaction in the corresponding external payment service before approval.

7.3 Approval transaction

Approval must be idempotent and transactional. A double click or repeated request must not allocate slots twice or send duplicate state changes.

```
BEGIN
  lock/read registration and verify status=PENDING_REVIEW
  verify pending game-entry reservations still valid
  payment.status = VERIFIED
  registration.status = CONFIRMED
  each registration_game_entry.status = CONFIRMED
  finalize reserved slots
  write audit log(actor=admin, action=REGISTRATION_APPROVED)
COMMIT

after commit: send approval email + .ics; record notification result
```

7.4 Rejection transaction

- Rejection reason is required and stored.
- Registration and payment states change to rejected, pending game entries are rejected, and reserved capacity is released atomically.
- Rejection email includes a participant-safe reason. Internal notes are never sent.
- Old rejected records remain available for audit/history. A participant may submit a new registration later if registration is still open and policy permits; duplicate rules must ignore rejected historical attempts where appropriate.

8. Tournament, Games and Participant Model

8.1 Terminology

Term	Definition
Tournament	One NDCAK event instance, e.g., Indoor Games Championship 2026.
Game	Reusable game/sport type, e.g., Chess or Table Tennis.
Tournament Game	Configuration of one game inside one tournament: fee, capacity, format, scoring adapter, rules, status.
Participant	Student identity. Can have registration history across tournaments.
Registration Game Entry	A participant's selected/confirmed entry for a specific tournament game.
Round	A stage within a tournament game (Round 1, Quarter Final, Group A, etc.).
Match	One scheduled contest. Can contain two OR more match entries.

Term	Definition
Match Entry	Link between a match and one competitor registration entry, with seat/order/result/placement.

8.2 Game configuration

- Admin can enable/disable a game for registration.
- Per game: capacity, fee, rules, registration open flag, tournament format, progression mode and scoring adapter.
- Maximum games per participant is a tournament-level configuration, not hardcoded to any number.
- Each tournament game may use AUTOMATIC progression or MANUAL progression. This is important for custom/multi-player formats such as card games where a standard 1v1 bracket may not fit.

8.3 Capacity semantics

- Capacity should count confirmed + pending reserved entries unless the committee chooses a different policy.
- Rejected/cancelled entries release capacity immediately.
- Capacity checks must happen inside database transactions to prevent race-condition oversubscription.
- Admin may increase/decrease capacity, but the system must block decreasing below current reserved/confirmed count unless an explicit override workflow is implemented.

9. Score Operator Accounts and Flexible Assignments

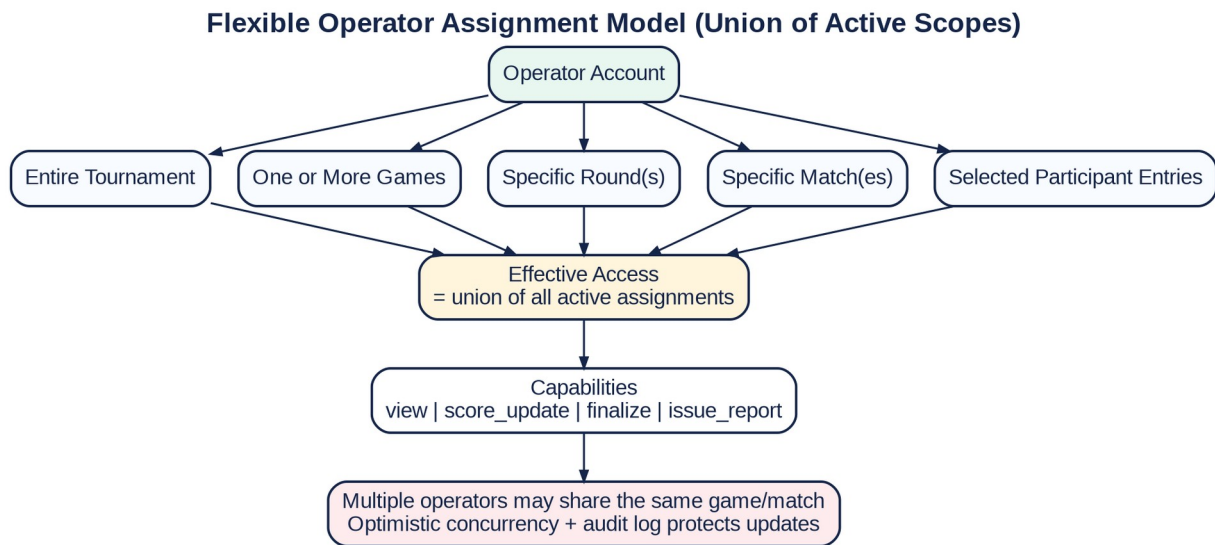


Figure 3. Flexible operator assignment model.

9.1 Account lifecycle

- Super Admin creates an operator account. Recommended flow: create staff profile + Supabase Auth invite/temporary credential.
- Operator role is SCORE_OPERATOR; role alone grants no tournament resources until assignments exist.
- Admin can deactivate an account or revoke individual assignments without deleting history.
- Operator identity is recorded on every score/finalization action.

9.2 Assignment scopes

Scope	Example	Effective access
ALL_TOURNAMENT	Senior operator	All games/rounds/matches in the tournament.
GAME	Chess + FIFA Mobile	All current/future matches under selected games.
ROUND	Table Tennis Semi Finals	Matches in selected round(s).
MATCH	FIFA Match M-041	Only explicitly assigned match(es).
PARTICIPANT_ENTRY	Selected chess participants	Matches involving the selected registration game entry/entries.

Multiple assignment rows are additive (union semantics). Multiple operators may have overlapping access to the same game or match. Explicit deny rules are not required in V1; revoke or narrow assignments instead.

9.3 Capabilities

- VIEW: see assigned game/match/participant operational data.
- SCORE_UPDATE: enter/update in-progress scoring data.
- FINALIZE_MATCH: submit a final result if allowed.
- ISSUE_REPORT: flag a match/problem for admin attention.
- Admin always has override capability. For V1, an operator assignment may simply include all standard operator capabilities unless the committee needs finer control.

9.4 Operator dashboard UX

- Landing page shows assigned games and next/active/awaiting-entry matches, not unrelated tournament-wide analytics.
- Provide search/filter by assigned game, match status, round and participant name/registration code.
- Primary action from the list is "Open score entry".
- On mobile/tablet, use large action targets and keep score input above secondary information.
- Operator should never need to navigate to an admin-style registrations or payments table.

10. Match Operations and Game-Specific Scoring

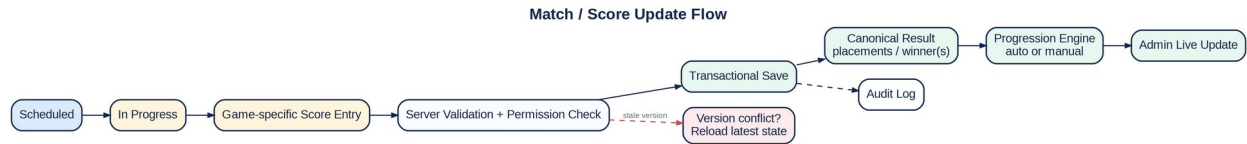


Figure 4. Score submission, validation, persistence and progression.

10.1 Match model must support N competitors

Do not model matches as player_1_id + player_2_id

Chess/FIFA may be 1v1, but the system requirement explicitly allows multiple players in a game/match. Use a match_entries junction table so a match can contain 2, 3, 4 or more competitors.

10.2 Match statuses

Status	Meaning
SCHEDULED	Created with participants/time/round; not started.
IN_PROGRESS	Score entry is active.
COMPLETED	Final result accepted.
WALKOVER	Result determined because competitor did not play.
POSTPONED	Temporarily delayed/rescheduled.
CANCELLED	Match cancelled by authorized staff.

10.3 Game-specific scoring adapters

Implement a TypeScript scoring-adapter interface. Each tournament game references an adapter key. The UI and server share adapter-specific validation, while the database stores canonical result/placement output plus structured JSON for game-specific details.

Game	Example V1 score UI / result data
Chess	Outcome selection (competitor A win / draw / competitor B win) plus optional note. If tournament rules forbid draws, adapter enforces tie-break state.
FIFA / PES Mobile	Regular goals, optional extra-time score, optional penalty score, final winner.
Table Tennis	Set-by-set score; server determines sets won and winner according to configured best-of format.
Carrom	Configurable frame/point result based on committee rules; avoid hardcoding before rule finalization.
29 Cards	Support multi-player entry and configurable points/placement/result fields. Manual progression may be used if bracket logic is non-standard.

10.4 Canonical result contract

```

type CanonicalMatchResult = {
  matchId: string;
  status: 'COMPLETED' | 'WALKOVER';
  placements: Array<{
    matchEntryId: string;
    placement: number;
    points?: number;
    outcome?: 'WIN' | 'LOSS' | 'DRAW' | 'DQ' | 'DNS';
  }>;
  winnerEntryIds: string[];
  displayScore: string;
  resultData: Record<string, unknown>; // adapter-specific validated JSON
};

```

10.5 Finalization rules

- Server verifies operator effective access at request time; UI visibility is not authorization.
- Server validates match status and expected record version before applying updates.
- Final result is written in a database transaction with canonical placements, match status, progression updates and audit log.

- After a match is COMPLETED, normal operator edits should be disabled. Admin can reopen/correct with a mandatory audit reason; optionally grant a specific operator correction permission later.
- If multiple operators are assigned, use optimistic concurrency (version column) so a stale editor cannot silently overwrite a newer result.

11. Tournament Progression and Results

11.1 Progression modes

Mode	Behavior
AUTOMATIC_SINGLE_ELIMINATION	On finalization, winning/qualifying entry is inserted into the configured next match/round slot.
MANUAL	Result is stored, but admin chooses/creates the next-round competitors. Useful for custom multi-player/card formats.
FUTURE: ROUND_ROBIN / GROUP	Data model may support later; not required to block V1.

11.2 Bracket / round generation

- Admin can generate or manually create rounds/matches after registration closes or when participant list is ready.
- Seeding strategy can be random/manual in V1. Do not claim advanced ranking seeding unless implemented.
- Bracket generation must operate on CONFIRMED registration_game_entries only.
- Byes and uneven participant counts must be supported for single elimination.
- Admin can edit schedule/venue/table metadata without altering result history.

11.3 Multi-event awards / cross-game points

The platform may calculate tournament-wide awards such as highest cumulative points or most game segments. Award rules must be configurable rather than embedded in UI. For example, the committee may configure champion=3 points and runner-up=1 point. Calculations derive from finalized game placements only.

- Never calculate awards from in-progress scores.
- Tie-break policy must be explicit/configurable (for example number of wins, then runner-up finishes, then admin decision).
- Admin can view the computed leaderboard. Public visibility is a separate setting.

12. Super Admin Operations, Analytics and Reporting

12.1 Admin navigation (recommended)

Module	Core functions
Dashboard	Operational summary, pending approvals, active matches, recent operator activity, key counts.
Registrations	Pending review, confirmed/rejected history, search/filter/pagination, review detail.
Participants	Confirmed participant directory and game entries; no separate participant account management.
Payments	Verification history/search; optionally combined with Registrations if separate module adds no value.
Games & Event	Tournament settings, tournament games, capacities, fees, rules, schedule, rounds/matches.
Operators	Create/deactivate operator accounts; see workload and activity.
Assignments	Grant/revoke tournament/game/round/match/participant scopes.
Matches / Scores	Monitor all matches, reopen/correct results, resolve issues.
Reports & Analytics	Registration/game counts, completion stats, revenue totals, result summaries, export.
Audit Log	Searchable immutable activity trail for critical actions.
Settings	Association/event content, payment methods, email settings, registration limits.

12.2 Dashboard metrics

- Pending registrations / payment verification.
- Confirmed registrations and total confirmed game entries.
- Capacity utilization per game.
- Active / scheduled / completed matches.
- Active operators and assignments.

- Expected/verified registration fee totals (do not label as accounting-grade revenue unless that is truly supported).
- Recent score updates and issue reports.

12.3 Reporting/export

- CSV export for registrations, confirmed participants by game, payment verification, match results and operator activity.
- Exports must respect the active admin filters where practical.
- Large exports should be generated server-side; never construct from only the current browser page and call it “all data”.

13. Notifications and Email Automation

Trigger	Recipient	Message / attachment
Payment/registration submitted	Participant	Receipt of submission; pending verification; registration code; selected games; no .ics.
Admin approved	Participant	Registration confirmed; selected games; event/check-in details; .ics calendar invite.
Admin rejected	Participant	Rejected/not confirmed; participant-safe rejection reason; organizer contact guidance.
Scheduled reminder	Confirmed participants	Event reminder (for example configured number of days before event).
Schedule change (P1)	Affected confirmed participants	Updated time/venue details.
Operator account created	Operator	Invite or temporary credential / password setup link.

13.1 Reliability

- Email sending occurs after the database transaction commits. A failed email must not roll back an approved registration.
- Record notification status (queued/sent/failed), provider message ID where available, error text and retry count.
- Admin can manually retry a failed confirmation/rejection email.
- Reminders use a scheduled server job (Vercel Cron or equivalent) and must be idempotent to avoid duplicates.

14. Pagination, Search, Filtering and Data Management

System-wide requirement

Every list that can grow significantly must use server-side query controls. Do not fetch hundreds/thousands of registrations/matches and filter only in React.

Dataset	Default page size	Core filters
Registrations	25	status, game, department, payment method, date range, search
Participants	25	game, department/year, confirmation status, search
Matches	25	game, round, status, date, operator, search
Operators	25	active status, assigned game, search
Audit logs	50	actor, action type, entity type, date range, search
Notifications	25	type, delivery status, date, recipient search

- Keep filter/search/page state in URL query parameters for shareable/reload-safe admin views.
- Debounce free-text search around 250-400ms or submit explicitly; avoid one request per keystroke.
- Use deterministic sort keys (for example created_at DESC, id DESC) to avoid duplicate/missing rows across pages.
- Offset pagination is acceptable at this event scale for admin page-number UX. Cursor pagination is recommended for long append-only audit logs if needed.
- Provide empty states and clear “reset filters” actions.

15. Responsive, Accessibility and Performance Requirements

15.1 Accessibility

- Target WCAG 2.1 AA contrast for text and interactive controls.
- Every form field has a persistent label; placeholder text is not the only label.
- Keyboard navigation and visible focus states for all controls.
- Never communicate status by color alone; include text/icon labels such as Confirmed, Pending, Rejected.
- Error messages should identify what is wrong and how to fix it.
- Tables must remain readable and headers should be semantically marked.

15.2 Performance

- Use Next.js Server Components for data-heavy read views where appropriate; keep client components focused on interaction.
- Use Next/Image or equivalent optimized image delivery, AVIF/WebP when possible, explicit sizes and lazy loading for non-critical assets.
- Public informational pages should be cached/revalidated because schedule/rules change infrequently.
- Staff data queries must select only needed columns and use indexes for status, tournament/game IDs, created_at, registration code, student ID and transaction ID lookups.
- Avoid N+1 queries on registration/game/match list pages. Prefer joins/aggregations or batched queries.
- Realtime subscriptions are primarily for authenticated staff. Public live results (if enabled) should use periodic refresh/revalidation rather than one realtime socket per visitor by default.
- Provide route-level loading states/skeletons, but do not mask slow queries with excessive animation.

16. Technical Architecture and Stack

Layer	Decision	Reason
Web framework	Next.js App Router + TypeScript	Unified public/staff app, server rendering, route handlers/actions, mature deployment on Vercel.
UI	Tailwind CSS + shadcn/ui + Lucide	Fast, accessible components while preserving custom NDCAK design system.
Forms/validation	React Hook Form + Zod	Efficient forms with shared client/server validation schemas.
Database	Supabase PostgreSQL (Free initially)	Relational model fits registrations, assignments, matches and audit; expected scale is small.
ORM	Drizzle ORM + Drizzle Kit	Lightweight, type-safe, SQL-friendly and suitable for serverless Postgres.
Staff auth	Supabase Auth	Email/password/invite for Super Admin and Score Operators only.
Realtime	Supabase Realtime	Near-live staff dashboard updates without a second infrastructure provider.
Email	Resend (+ React Email optional)	Transactional submission/approval/rejection/reminder emails.
Hosting	Vercel	Natural fit for Next.js; free tier target.
DNS/CDN	Cloudflare optional	DNS, caching/security controls if NDCAK domain is managed there.

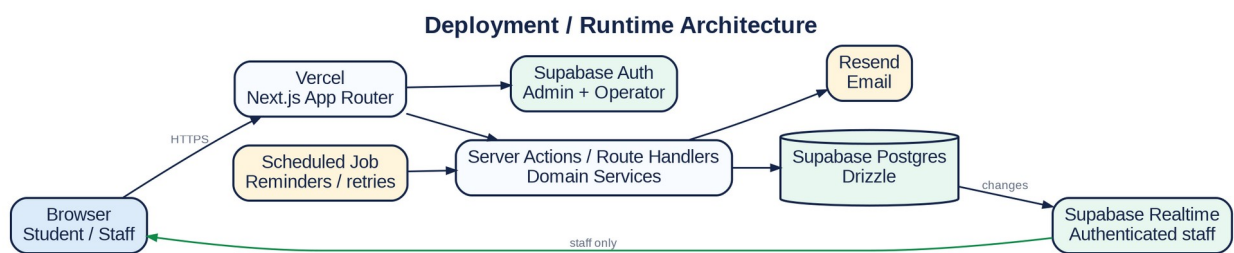


Figure 5. Deployment and runtime architecture.

16.1 Architectural boundaries

- Browser never receives Supabase service-role key or direct privileged database credentials.
- All public writes and all staff mutations go through trusted Next.js server code.
- Drizzle owns application database schema/migrations and server query logic.
- Supabase Auth handles staff identity; application authorization resolves role and resource scopes.
- Realtime is a delivery mechanism, not the source of truth. PostgreSQL remains authoritative.

17. Database Design

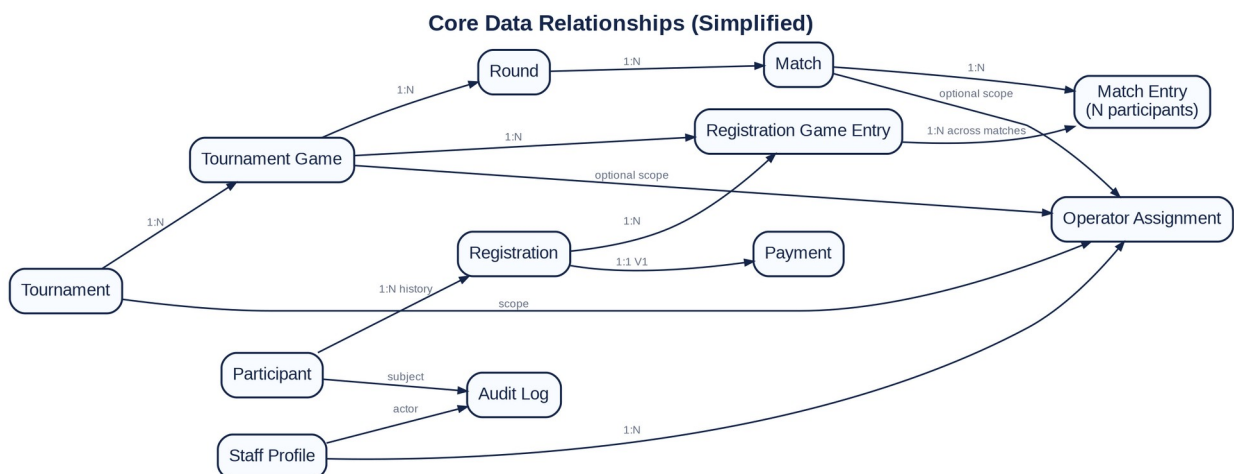


Figure 6. Simplified core data relationships.

17.1 Core tables

Table	Purpose / key fields
tournaments	id, name, slug, year, registration_open_at/close_at, start/end, venue, max_games_per_participant, status, public settings.
games	Reusable game catalog: id, name, slug, icon_key, description.
tournament_games	tournament_id, game_id, fee, capacity, status, scoring_adapter,

Table	Purpose / key fields
participants	progression_mode, rules, config_json. id, normalized_student_id, name, email, phone, department, academic_year, created_at.
registrations	id, code, tournament_id, participant_id, status, total_fee, submitted_at, reviewed_by, reviewed_at, rejection_reason.
registration_game_entries	id, registration_id, tournament_game_id, status, seed, created_at.
payment_methods	tournament_id, provider, display_name, receiving_account, instructions, enabled, sort_order.
payments	registration_id, provider, receiving_account_snapshot, expected_amount, transaction_id_normalized, transaction_id_raw, status, verified_by/at.
staff_profiles	auth_user_id, role, display_name, active.
operator_assignments	operator_id, tournament_id, scope_type, optional tournament_game_id/round_id/match_id/registration_game_entry_id, capabilities, active.
rounds	tournament_game_id, name, sequence, format/meta, status.
matches	round_id, code, status, scheduled_at, venue/table, result_json, display_score, winner metadata, version.
match_entries	match_id, registration_game_entry_id, seat/order, placement, outcome, score/result metadata.
match_updates	match_id, actor_staff_id, update_type, payload_json, created_at (optional granular scoring history).
notifications	registration/staff recipient refs, type, status, provider_id, error, sent_at, retry_count.
audit_logs	actor, action, entity_type/id, before_json, after_json, reason, created_at, request metadata as appropriate.

17.2 Critical constraints / indexes

- Unique tournament registration code.
- Prevent more than one active CONFIRMED/PENDING registration for the same participant+tournament as defined by policy; rejected history must not permanently block resubmission.
- Unique registration_game_entry per registration+tournament_game.
- Index registrations(tournament_id,status,submitted_at), participants(normalized_student_id), payments(provider,transaction_id_normalized), matches(round_id,status,scheduled_at), operator_assignments(operator_id,active).
- Use database/check constraints where possible for status enums and valid amounts/capacities.
- Capacity enforcement cannot rely only on a count query outside a transaction; protect with row locking/advisory strategy or a maintained reservation counter in a transaction.

18. Authorization, Security, Concurrency and Auditability

18.1 Authorization model

Use RBAC for coarse role (SUPER_ADMIN vs SCORE_OPERATOR) plus resource-based assignment checks for operators. This is effectively RBAC + ABAC/resource scopes.

```
canOperatorAccessMatch(operatorId, matchId, capability):
  return active assignment exists where
    scope == ALL_TOURNAMENT
    OR scope game == match.tournament_game_id
    OR scope round == match.round_id
    OR scope match == match.id
    OR scope participant_entry intersects match.entries
  AND assignment grants capability
```

- Every server mutation repeats authorization. Never rely on hidden buttons/client route guards only.
- Super Admin bypasses operator scope checks but still generates audit events.
- Any client-side Supabase Realtime subscription should use authenticated channels/RLS as appropriate; do not expose privileged write access to the browser.

18.2 Security controls

- Zod validation on server for every external input.
- Cloudflare Turnstile or equivalent anti-bot/rate limiting on public registration submission (recommended P0/P1).
- Rate limit repeated registration attempts and staff login endpoints using platform-appropriate controls.
- Use secure, HTTP-only auth cookies/session handling supplied by Supabase/Next integration.
- Keep DB URLs, service-role key and Resend key server-only.
- Escape/render participant-entered data safely; never inject into HTML emails without templating/escaping.
- No sensitive values in application logs. Transaction IDs may be logged only in protected structured audit where needed, not console traces.
- Admin/operator routes should be noindex and require auth on the server.

18.3 Concurrency

- Registration approval/rejection uses row/state checks so the same request cannot be processed twice.
- Match record has integer version. Operator opens version N; update succeeds only if current version is still N, then increments to N+1.
- If version conflict occurs, return 409-style conflict and ask the operator to reload current match state rather than overwriting.
- Optional P1: show staff presence/"being edited by" hint, but correctness must not depend on a UI lock.

18.4 Audit requirements

- Audit registration approved/rejected/cancelled, payment status changes, operator account create/deactivate, assignment add/revoke, match finalize/reopen/correct, game configuration changes and manual progression overrides.
- Audit records are append-only to application users; do not offer a normal "delete audit record" UI.
- For corrections, store a reason and meaningful before/after snapshot.

19. Application Routes and Screen Specifications

19.1 Public routes

Route	Screen requirements
/	Hero with event essentials and one primary registration CTA; games overview; deadline/schedule/prize summary; clean footer.
/register	Form + game selection; fee/capacity; inline validation; no unrelated marketing clutter after form starts.
/register/review	Read-only summary; edit/back; proceed to payment.
/register/payment	Expected amount, method cards/select, receiving account/instructions, transaction ID; final submit.
/register/submitted?code=...	Pending verification success state; same registration code; summary; submission email sent. No confirmed language.
/schedule	Day/game filters; accessible schedule list/table.

Route	Screen requirements
/rulebook	General + game-specific navigation/accordion/side nav depending viewport.
/results	Optional public live/completed information controlled by config.
/staff/login	Minimal staff-only login; redirect by role.

19.2 Admin routes

Route	Purpose
/admin	Operational dashboard.
/admin/registrations	Paginated registration queue/history.
/admin/registrations/[id]	Full review and approval/rejection detail.
/admin/participants	Confirmed participant directory by game.
/admin/games	Tournament game configuration and capacities.
/admin/games/[id]	Rules, scoring adapter, rounds/matches, schedule.
/admin/operators	Operator accounts and status.
/admin/operators/[id]	Profile, assignments, workload and activity.
/admin/assignments	Bulk/flexible assignment management.
/admin/matches	Tournament-wide match monitor.
/admin/matches/[id]	Result detail, audit, reopen/correct.
/admin/reports	Analytics and exports.
/admin/audit	Audit log.
/admin/settings	Event/payment/communication configuration.

19.3 Operator routes

Route	Purpose
/operator	Assigned workload dashboard.
/operator/games	Only assigned games/participant scopes.
/operator/matches	Paginated/filtered assigned match queue.
/operator/matches/[id]	Game-specific score-entry screen.
/operator/activity	Own recent score updates/issue reports (P1).

20. Codebase Architecture and Engineering Conventions

Use one Next.js repository with feature-oriented domain modules. Avoid a huge “utils” folder and avoid placing all DB logic directly inside page components.

```
src/
  app/
    (public)/
      page.tsx
      register/
      schedule/
      rulebook/
      results/
      staff/login/
      admin/
      operator/
      api/                # only where route handlers are appropriate
  components/
    ui/                  # shadcn primitives
    brand/               # logo, header, public layout primitives
    data-table/
  features/
    registration/
      schemas.ts
      server/
      components/
    payments/
    tournaments/
    games/
    matches/
    scoring/
      adapters/
    operators/
    assignments/
    notifications/
    analytics/
  db/
    index.ts
    schema/
    migrations/
  lib/
    auth/
    permissions/
    realtime/
    email/
    config/
  styles/
  types/
  tests/
    unit/
    integration/
    e2e/
```

20.1 Domain service rules

- Pages/components call feature-level server actions/services; they do not contain tournament/business rules.
- Scoring adapters own game-specific validation and canonical result conversion.
- Permission resolution lives in one shared module and is used by every operator mutation/read service.
- Registration fee/capacity calculation lives server-side in a domain service and is reused by review/submission/admin views.
- Database migrations are committed to source control. Never mutate production schema manually without corresponding migration.

20.2 Recommended packages

Need	Package / approach
Validation	zod
Forms	react-hook-form + @hookform/resolvers
Tables	TanStack Table underneath reusable admin table components, or shadcn patterns
Icons	lucide-react
Email templates	React Email optional + Resend
Dates	date-fns or native Temporal when project/runtime policy supports it
Testing	Vitest + Testing Library + Playwright

Need	Package / approach
Lint/format	ESLint + Prettier (or project-approved Biome, but choose one consistent toolchain)

21. API / Server Action Contract Guidance

Exact endpoint naming can evolve, but the coding agent should implement explicit typed commands rather than generic CRUD for critical workflows.

Command / query	Essential behavior
submitRegistration(input)	Validate event window, selected games/capacity, compute fee, normalize payment txn, create pending registration atomically.
approveRegistration(id)	Admin-only, pending-state check, verify/finalize payment and entries atomically, audit, then email.
rejectRegistration(id, reason)	Admin-only, release reservations atomically, audit, then email.
createOperator(input)	Admin-only; create auth user/invite + staff profile.
setOperatorAssignments(operatorId, assignments)	Admin-only; validate target resources, replace/add/revoke assignments, audit.
listAssignedMatches(operator, filters, page)	Resolve effective access server-side and paginate.
updateMatchScore(matchId, adapterPayload, version)	Authorized operator/admin; validate adapter payload + version; save update/audit.
finalizeMatch(matchId, adapterPayload, version)	Transactional final result + progression + audit + realtime.
reopenMatch(matchId, reason)	Admin-only; protect downstream dependencies; audit.
getAdminRegistrationPage(filters, page)	Server-side search/filter/count/list.

21.1 Error contract

- Return field-level validation errors for participant forms.
- Use explicit domain errors: REGISTRATION_CLOSED, GAME_FULL, DUPLICATE_TRANSACTION, ALREADY_REVIEWED, UNAUTHORIZED_SCOPE, STALE_MATCH_VERSION, INVALID_SCORE, DOWNSTREAM_RESULT_DEPENDENCY.
- User-facing text must be friendly; internal stack traces never reach the browser.

22. Testing and Quality Strategy

22.1 Unit tests (P0)

- Fee calculation and max-game validation.
- Capacity reservation/release decisions.
- Permission resolution across all assignment scopes and combinations.
- Each game scoring adapter validation and canonical result conversion.
- Tournament progression logic including byes and manual mode.
- Registration/match state-transition guards.
- Email payload generation and .ics creation.

22.2 Integration tests (P0)

- Concurrent registration submissions for the last available slot do not oversubscribe.
- Approve and reject transactions update all related records consistently.
- Duplicate transaction ID policy works after normalization.
- Operator can access a match via game/round/match/participant scope and cannot access an unrelated match.
- Match finalization writes result, increments version, creates audit and advances only when progression mode allows.
- Stale version update is rejected without overwriting newer score.

22.3 End-to-end tests (P0)

- Participant completes form -> review -> payment -> pending screen; acknowledgment email is triggered.
- Admin finds pending request, approves -> confirmation state/email/ICS; selected game entries confirmed.
- Admin rejects -> reservations released and rejection email created.
- Admin creates operator and assigns multiple games; operator sees only correct matches.
- Two operators overlap on a game; one finalizes match and the stale second submission is rejected/reloaded.
- Responsive registration flow works at common mobile viewport; operator score entry works on tablet/mobile; admin is usable on mobile.

22.4 Performance / pre-event rehearsal

- Seed realistic participant/match volume and test paginated admin pages.
- Rehearse the complete event-day flow with multiple operator accounts before registration closes.
- Test weak-network behavior and retry UX on score entry.
- Export a database backup before the event and verify restore/documented recovery process.

23. Deployment, Environments and Operations

23.1 Environments

Environment	Purpose
Local	Developer machine with local env vars / development Supabase project if available.
Preview	Vercel preview per branch/PR; must not point to production DB by default.
Production	Final domain, production Supabase project, Resend verified sender, production secrets.

23.2 Environment variables (illustrative)

```

DATABASE_URL=...
NEXT_PUBLIC_SUPABASE_URL=...
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=... # server only, only if required
RESEND_API_KEY=...
EMAIL_FROM=...
NEXT_PUBLIC_APP_URL=...
TURNSTILE_SITE_KEY=... # if enabled
TURNSTILE_SECRET_KEY=... # server only

```

23.3 Production checklist

- Run migrations and seed only reference/config data intentionally.
- Create at least two trusted Super Admin accounts if organizational policy allows, to avoid single-person lockout.
- Configure payment methods and verify receiving numbers manually.
- Verify association email sender/domain and deliverability.
- Confirm registration open/close time zone; store timestamps in UTC and display Asia/Dhaka for event UI.
- Run full approval/operator/match rehearsal against production-like data.
- Confirm Supabase project is active before event week; do not discover free-project inactivity behavior on event day.
- Have database export/backup and emergency admin procedure documented.

24. Delivery Phases

Phase	Scope	Exit condition
0. Foundation	Next.js repo, design tokens, Supabase/Drizzle schema/migrations, auth skeleton, environments.	App deploys; migrations repeatably run; staff auth works.
1. Public + Registration	Home/schedule/rules shell, registration/review/payment/submitted, capacity/fee logic, acknowledgment email.	Complete participant flow ends in pending request.
2. Admin Verification	Admin dashboard, paginated registrations, review detail, approve/reject transactions, confirmation/rejection emails + ICS.	Registration lifecycle production-ready.
3. Operators + Assignments	Create operator, staff auth, flexible assignments, assigned workload queries.	Operators see only intended resources.
4. Matches + Scoring	Rounds/matches, match_entries, scoring adapters, score entry, audit, concurrency, progression.	Multiple operators can safely score configured games.
5. Realtime + Analytics	Staff realtime updates, operational dashboard metrics, exports, reminders.	Event-day monitoring and communication ready.
6. Hardening	Responsive QA, accessibility, load/concurrency tests, backups, runbook, bug fixing.	Production rehearsal passes.

25. Acceptance Criteria

25.1 Participant / registration P0

- A student can complete registration from a mobile phone without creating an account.
- All game choices occur during the initial form; selected games and total fee are visible before payment.
- Final submission cannot confirm registration automatically; it creates a pending verification request.
- Pending screen/email uses pending wording and displays a stable registration code.
- Admin approval changes the whole registration and its game entries to confirmed atomically.
- Admin rejection stores a reason, releases capacity, and triggers rejection communication.
- Approval email includes final event/check-in information and calendar invite.

25.2 Admin P0

- Admin can search/filter/paginate registrations and review all payment fields needed for manual verification.
- Admin can create/deactivate Score Operator accounts.
- Admin can assign one operator to multiple games and multiple operators to one game.
- Admin can assign full-tournament, game, round, match or participant-entry scopes.
- Admin can see match/result changes near real time and can inspect audit history.
- Admin can correct/reopen a result with a recorded reason and safely handle downstream progression.

25.3 Operator P0

- Operator sees only assigned game/match/participant scope.
- Operator can score supported game types using appropriate game-specific input UI.
- Match model supports more than two competitors.
- Score finalization is validated on server and recorded with actor/time/audit.
- Stale concurrent updates cannot overwrite newer match state silently.
- Operator UI is usable on phone/tablet and can find assigned work quickly.

25.4 Technical P0

- All production DB changes are managed by Drizzle migrations.
- All sensitive writes pass through server authorization and validation.
- All growing admin tables use server-side query pagination/filtering.
- Primary public pages are responsive, accessible and optimized; staff pages are noindex.
- No secrets/service credentials are exposed to client bundles.
- Critical workflows have unit/integration/E2E coverage before event day.

26. Edge Cases and Failure Handling

Scenario	Required behavior
Two students compete for final slot	Transactional reservation allows at most one valid reservation; the other receives GAME_FULL and no partial records.
Same student submits twice	Block active duplicate pending/confirmed attempt per policy; allow resubmission after rejection if registration remains open.
Same transaction ID reused	Block or flag duplicate normalized provider+transaction ID; admin should see previous reference.
Admin double-clicks Approve	Idempotent state check; second request returns already reviewed, no duplicate side effects.
Registration closes during review/payment	Server final submit checks close time; show clear expired message.
Admin changes payment number later	Existing payment row retains receiving_account_snapshot shown at submission.
Operator loses assignment while page open	Next mutation fails authorization; prompt reload.
Two operators edit same match	Version check prevents stale overwrite; latest state reload.
Wrong final score submitted	Admin reopens/corrects with reason; dependent next-round state must be reconciled transactionally.
Participant no-show	Authorized staff marks walkover/DNS according to game rules; audit it.
Match postponed	Keep participants; change status/time; notify affected users only if notification feature enabled.
Multi-player match tie	Scoring adapter requires tie-break/placement policy or keeps match unfinalized until resolved.
Email provider failure	DB state remains correct; notification marked failed and retryable.
Realtime disconnected	Operator/admin can still perform server writes/refresh; realtime is enhancement, not correctness dependency.
Free DB/service temporarily unavailable	Show recoverable error; do not duplicate submission on retry; preserve idempotency keys where appropriate.
Game removed after registrations exist	Block destructive deletion; use inactive/archived status and require explicit migration/cancellation process.

27. Future Extensions and Explicit Non-Goals

27.1 P1 / P2 extensions

- Public live-results / bracket viewer with controlled refresh.
- QR-based check-in using confirmed registration code.
- Automated participant match reminders.
- Waitlist when a game reaches capacity.
- Team/partner entities for doubles/team formats.
- Round-robin/group-stage engine and richer seeding algorithms.
- Payment gateway/API verification if the association later obtains supported merchant integration.
- Certificate generation and downloadable result certificates.
- Multi-year tournament archive and historical participant statistics.
- Fine-grained operator capabilities beyond standard score/update/finalize workflow.

27.2 Avoid premature complexity

Do not add Redis, microservices, a message broker, Kubernetes, a separate backend framework, participant accounts, or a custom websocket server merely because they are common in larger systems. The first event is small enough for a well-structured Next.js + PostgreSQL monolith. Add infrastructure only when measured needs justify it.

28. Implementation Checklist and Definition of Done

28.1 Agent start checklist

1. Create Next.js TypeScript repository and configure formatting/lint/testing.
2. Implement design tokens and public/staff layout shells before detailed screens.
3. Create Supabase projects/environments and Drizzle schema/migrations.
4. Implement staff auth + role/profile resolution.
5. Implement registration domain models, fee/capacity services and public three-step flow.
6. Implement manual payment submission and admin approval/rejection transaction.
7. Implement email templates and notification logging.
8. Implement tournament games/rounds/matches/match_entries without a two-player assumption.

9. Implement operator assignment resolver and admin operator management.
10. Implement scoring adapter interface and each approved game adapter.
11. Implement transactional finalization/progression + optimistic concurrency + audit.
12. Add realtime staff subscriptions after correctness works with normal refresh.
13. Add pagination/filtering/exports and final responsive/accessibility/performance pass.
14. Run production rehearsal with realistic seeded data and multiple operator accounts.

28.2 Definition of Done

Production-ready means more than “screens exist”

The first event is ready only when registration states, manual payment approval, slot integrity, operator authorization, multi-player match data, game-specific result validation, concurrency protection, audit logs, responsive UX, notification failure handling, database migrations, tests and event-day rehearsal all pass together.

Final product statement

NDCAK Digital Tournament Management System is a clean, responsive, role-based event operations platform. Participants register once and then leave the complexity behind. Super Admin verifies registrations and controls the tournament. Score Operators work only inside flexible assigned scopes. PostgreSQL is the source of truth; game-specific adapters preserve real scoring rules; every critical state transition is validated and auditable. The product should feel like a polished NDCAK event website on the public side and a dependable operational tool behind the scenes.